

多轮对话对大模型文本蕴含能力的增强效果：基于海龟汤游戏的实证研究

摘要

在本次作业中，笔者主要探讨了大型语言模型（LLMs）在多轮对话中处理文本蕴含任务的能力，以及对话历史对其性能的影响。笔者以中文推理游戏"海龟汤"为测试场景，构建了一个包含30个谜题的数据集，每个谜题配有5-8轮问答的思考链。笔者设计了两组实验，对比了三个主流大模型（Claude 3.7、GPT-4o和ChatGLM4）在有无对话历史条件下的表现。实验结果表明，模型在处理特定类型的谜题（如涉及隐含社会关系的"闺蜜"谜题和包含日常物品特殊用途的"洗衣机"谜题）时仍面临挑战，但对话历史显著提升了模型的文本蕴含能力，Claude 3.7的准确率从92.8%提升至96.6%，GPT-4o从94.9%提升至94.1%，ChatGLM4从84.8%提升至92.8%。

1. 引言

随着大型语言模型（LLMs）的快速发展，其在各类自然语言理解任务上展现出了惊人的能力。然而，在需要深度推理和上下文理解的场景中，LLMs的表现仍有提升空间。文本蕴含（Textual Entailment）作为自然语言处理的基础任务之一，要求模型判断一个文本片段（前提）是否能推导出另一个文本片段（假设）。这种能力对于问答系统、信息抽取和对话理解等应用至关重要。

传统的文本蕴含任务通常是静态的单轮评估，无法完全反映实际应用中的多轮交互特性。在真实的对话场景中，对话历史往往包含重要的上下文信息，可能显著影响模型对当前轮次的理解和推理。然而，关于对话历史如何影响LLMs处理文本蕴含任务的研究相对匮乏。

本文选择"海龟汤"游戏作为研究场景，这是一种源自日本的推理游戏，玩家通过提问来推理出完整故事。在游戏中，主持人了解完整故事（"汤底"），而玩家只知道故事的开头（"汤面"）。玩家通过提问来推理出完整故事，主持人只能回答"是"、"不是"或"这个问题与解谜无关"。这一游戏机制与文本蕴含任务高度契合：给定前提（汤底）和问题（玩家提问），判断前提是否蕴含（entail）、矛盾（contradict）或中立（neutral）于问题的答案。

在本次作业中，笔者进行的主要工作包括：

- 构建了一个包含30个中文海龟汤谜题的数据集，每个谜题配有5-8轮的思考链，总计约240个问答对，为研究多轮对话中的文本蕴含提供了资源。
- 设计了两组对比实验，系统评估了三个主流大模型（Claude 3.7、GPT-4o和ChatGLM4）在有无对话历史条件下的文本蕴含能力。
- 发现并分析了对话历史对大模型文本蕴含能力的积极影响，以及模型在处理特定类型谜题时的挑战和错误模式。

4. 探讨了多轮对话中的文本蕴含任务的特点及其对增强LLMs在实际应用中的推理能力的启示。

2. 背景信息

2.1 文本蕴含与自然语言推理

文本蕴含 (Textual Entailment) 是自然语言处理领域的一个基础研究问题,旨在确定一个文本片段是否能合理推导出另一个文本片段 (Dagan等, 2005)。随着研究的深入,这一任务逐渐扩展为自然语言推理 (Natural Language Inference, NLI) 任务,包括蕴含 (entailment)、矛盾 (contradiction) 和中立 (neutral) 三种关系判断 (Bowman等, 2015)。

早期的NLI数据集如SNLI (Stanford Natural Language Inference) 和MultiNLI提供了大规模的标注数据,推动了基于神经网络的方法的发展。这些数据集通常包含单轮的前提-假设对,缺乏多轮对话交互的特性。

2.2 大型语言模型的推理能力

大型语言模型如GPT系列、Claude系列和ChatGLM系列等在各种自然语言理解任务上展示了强大的能力。这些模型通过大规模预训练获取了丰富的世界知识和语言理解能力,在零样本和少样本学习场景下表现出色。

然而,关于LLMs推理能力的研究显示,这些模型在需要深度推理的任务上仍面临挑战。特别是在处理隐含信息、识别矛盾和进行多步推理时,LLMs的表现存在不稳定性。一些研究表明,思维链提示 (Chain-of-Thought prompting) 和上下文学习 (in-context learning) 可以显著提升模型的推理能力,这表明对话历史中的推理过程可能对提高模型性能有积极影响。

2.3 海龟汤作为多轮文本蕴含任务的可行性

海龟汤作为一种推理游戏,具有明确的评估标准 (回答的准确性) 和多轮交互的特性,为评估模型的推理能力提供了理想的场景。游戏规则要求参与者根据主持人给定的前提(汤底)和话题(汤面)编写回复,而主持人需要根据游戏规则判断参与者的回复是否合理,即参与者的回复是否可以从给定的前提中推理得出。这实际上是一个文本蕴含问题。通过在海龟汤的游戏机制中引入文本蕴含技术,可以增强游戏的互动性和挑战性,同时也为研究大规模语言模型的推理能力提供了一个有趣的视角。

3. 方法

3.1 任务定义

本研究将海龟汤游戏形式化为一个多轮文本蕴含任务。在这个任务中:

- 给定一个谜题 P , 包含表面描述 (汤面) S 和完整解释 (汤底) B 。
- 在第 t 轮对话中, 玩家提出问题 q_t 。

3. 系统需要根据完整解释 B 和先前的对话历史 $H_{t-1} = (q_1, a_1), (q_2, a_2), \dots, (q_{t-1}, a_{t-1})$ ，判断问题 q_t 的答案 $a_t \in \{\text{是}, \text{不是}, \text{这个问题与解谜无关}\}$ 。

笔者的研究目标是评估对话历史 H_{t-1} 对大型语言模型执行上述任务能力的影响。

3.2 数据集构建

为了系统评估大模型在海龟汤游戏中的表现，笔者构建了一个包含30个中文海龟汤谜题的数据集。每个谜题包含：

- 谜题标题
- 汤面 (surface)：故事开头，玩家可见
- 汤底 (bottom)：完整故事解释，玩家不可见
- 思考链 (thought_chain)：5-8轮的问答过程，模拟玩家逐步推理的过程

思考链是本数据集的核心部分，每轮包含：

- 问题 (question)：玩家提出的问题
- 答案 (answer)：基于汤底的标准答案，限于"是"、"不是"或"这个问题与解谜无关"

为确保思考链的质量和一致性，笔者首先使用Claude 3.7模型为每个谜题生成初始思考链，然后审核并修改，确保：

1. 每个问题明确且可回答
2. 答案基于汤底是准确的
3. 思考链展现出合理的推理进展

最终数据集包含约240个高质量的问答对，覆盖了多种推理类型，包括时间推理、空间关系推理、社会关系推理、物理因果推理等。

3.3 实验设计

笔者设计了两组实验来评估对话历史对大模型在海龟汤游戏中表现的影响：

实验1（无对话历史）：在这个实验中，模型仅基于汤面、汤底和当前问题作出判断，不考虑之前的问答历史。这模拟了静态的文本蕴含任务。

实验2（有对话历史）：在这个实验中，模型可以访问之前所有的问答对，模拟真实的多轮对话场景。

所有实验都在三个主流大模型上进行：

- Claude 3.7 (Anthropic)
- GPT-4o (OpenAI)
- ChatGLM4 (智谱AI)

为了确保实验的一致性，笔者为所有模型使用相同的系统提示（system prompt），指导模型扮演海龟汤游戏的主持人角色。

3.4 评估指标

笔者采用以下指标评估模型的表现：

- 1. **准确率（Accuracy）**：模型回答与标准答案匹配的比例
- 2. **分类别F1值**：对"是"、"不是"和"无关"三类回答分别计算F1值
- 3. **宏平均F1（Macro-F1）**：所有类别F1值的平均值
- 4. **微平均F1（Micro-F1）**：考虑类别不平衡的F1计算
- 5. **加权F1（Weighted-F1）**：按类别频率加权的F1值
- 6. **轮次准确率**：每一轮对话的准确率，用于分析模型表现随对话进展的变化

此外，笔者还提取并分析了模型的错误案例，以识别模型在处理特定类型问题时的挑战。

4. 实现细节

4.1 实验流程

笔者的实验流程包括以下步骤：

- 1. 加载海龟汤数据集
- 2. 分别在有无对话历史的条件下，让大模型对每个问题进行回答
- 3. 将模型回答分类为"是"、"不是"或"这个问题与解谜无关"
- 4. 与标准答案比较，计算评估指标
- 5. 提取和分析错误案例

以下是实验脚本的核心部分：

4.1.1 实验1（无对话历史）实现

▼ 代码块

```
1  def run_experiment(dataset_path: str, output_path: str, llm_provider:
    LLMProvider):
2      # 确保LLM提供者已初始化
3      llm_provider.initialize()
4
5      # 加载数据集
6      puzzles = load_dataset(dataset_path)
7
8      results = []
```

```

9
10     for puzzle in puzzles:
11         puzzle_results = {
12             "title": puzzle["title"],
13             "surface": puzzle["surface"],
14             "bottom": puzzle["bottom"],
15             "questions": []
16         }
17
18         # 创建系统提示
19         system_prompt = create_system_prompt(puzzle["surface"],
20 puzzle["bottom"])
21
22         # 处理思考链中的每个问题
23         for round_idx, round_data in enumerate(puzzle["thought_chain"]
24 ["rounds"]):
25             question = round_data["question"]
26             expected_answer = round_data["answer"]
27
28             # 检查问题格式, 跳过不正确格式的问题
29             if not isinstance(question, str) or question.strip() == "":
30                 print(f"跳过空问题或无效问题, 轮次: {round_idx + 1}")
31                 continue
32
33             # 分离问题文本和实际问题
34             question_lines = question.strip().split('\n')
35             actual_question = question_lines[-1] if question_lines else
36 question
37
38             # 使用LLM提供者发送问题
39             llm_response = llm_provider.ask_question(system_prompt,
40 actual_question)
41
42             print(f"轮次 {round_idx + 1}: 问题: {actual_question}")
43             print(f"LLM回应: {llm_response}")
44
45             # 分类回答
46             classified_response = classify_response(llm_response)
47
48             # 规范化预期答案以便准确比较
49             normalized_expected = normalize_answer(expected_answer)
50             normalized_classified = normalize_answer(classified_response)
51
52             # 记录结果
53             question_result = {
54                 "round": round_idx + 1,
55                 "question": question,
56                 "expected_answer": expected_answer,

```

```

52         "llm_response": llm_response,
53         "classified_response": classified_response,
54         "correct": normalized_classified == normalized_expected
55     }
56
57     print(f"预期答案: '{normalized_expected}', 分类答案:
58     '{normalized_classified}', 正确: {question_result['correct']}")
59     puzzle_results["questions"].append(question_result)
60
61     # 计算此谜题的准确率
62     correct_count = sum(1 for q in puzzle_results["questions"] if
63     q["correct"])
64     total_questions = len(puzzle_results["questions"])
65     puzzle_results["accuracy"] = correct_count / total_questions if
66     total_questions > 0 else 0
67
68     results.append(puzzle_results)
69
70     # 计算总体准确率
71     all_questions = [q for puzzle in results for q in puzzle["questions"]]
72     overall_accuracy = sum(1 for q in all_questions if q["correct"]) /
73     len(all_questions) if all_questions else 0
74
75     # 保存结果
76     with open(output_path, "w", encoding="utf-8") as f:
77         json.dump({
78             "puzzles": results,
79             "overall_accuracy": overall_accuracy
80         }, f, ensure_ascii=False, indent=2)
81     return overall_accuracy

```

在这个实现中，`create_system_prompt` 函数创建了指导模型扮演海龟汤主持人的系统提示，`classify_response` 函数将模型的自然语言回答分类为标准答案格式。每个问题都是独立处理的，没有考虑之前的问答历史。

4.1.2 实验2（有对话历史）实现

▼ 代码块

```

1  def run_experiment(dataset_path: str, output_path: str, llm_provider:
2      LLMProvider):
3      # 确保LLM提供者已初始化
4      llm_provider.initialize()
5
6      # 加载数据集
7      puzzles = load_dataset(dataset_path)

```

```

7
8     results = []
9
10    for puzzle in puzzles:
11        puzzle_results = {
12            "title": puzzle["title"],
13            "surface": puzzle["surface"],
14            "bottom": puzzle["bottom"],
15            "questions": []
16        }
17
18        # 创建系统提示
19        system_prompt = create_system_prompt(puzzle["surface"],
20        puzzle["bottom"])
21
22        # 处理思考链中的每个问题
23        for round_idx, round_data in enumerate(puzzle["thought_chain"]
24        ["rounds"]):
25            question = round_data["question"]
26            expected_answer = round_data["answer"]
27
28            # 构建前序对话记录
29            conversation_history = []
30            for prev_idx in range(round_idx):
31                prev_question = puzzle["thought_chain"]["rounds"][prev_idx]
32                ["question"]
33                prev_answer = puzzle["thought_chain"]["rounds"][prev_idx]
34                ["answer"]
35
36            # 确保对话历史中使用的答案格式正确
37            formatted_answer = format_answer_for_history(prev_answer)
38
39            # 添加到对话历史
40            conversation_history.append({"role": "user", "content":
41            prev_question})
42            conversation_history.append({"role": "assistant", "content":
43            formatted_answer})
44
45            # 使用LLM提供者发送问题，包含对话历史
46            llm_response = llm_provider.ask_question(system_prompt, question,
47            conversation_history)
48
49            print(f"回合 {round_idx + 1}, 问题: {question}")
50            print(f"原始期望答案: {expected_answer}")
51            print(f"标准化期望答案:
52            {normalize_expected_answer(expected_answer)}")
53            print(f"回答: {llm_response}")

```

```

46
47         # 分类回答
48         classified_response = classify_response(llm_response)
49
50         # 标准化期望答案
51         normalized_expected_answer =
normalize_expected_answer(expected_answer)
52
53         # 记录结果
54         question_result = {
55             "round": round_idx + 1,
56             "question": question,
57             "expected_answer": expected_answer,
58             "normalized_expected_answer": normalized_expected_answer,
59             "llm_response": llm_response,
60             "classified_response": classified_response,
61             "correct": classified_response == normalized_expected_answer,
62             "conversation_history": [
63                 {"role": m["role"], "content": m["content"]}
64                 for m in conversation_history if m["role"] == "user"
65             ] # 仅保存用户问题用于记录
66         }
67
68         puzzle_results["questions"].append(question_result)
69
70         # 计算此谜题的准确率
71         correct_count = sum(1 for q in puzzle_results["questions"] if
q["correct"])
72         total_questions = len(puzzle_results["questions"])
73         puzzle_results["accuracy"] = correct_count / total_questions if
total_questions > 0 else 0
74
75         results.append(puzzle_results)
76

```

与实验1相比，实验2的关键区别在于构建并使用了对话历史。对于每个问题，模型不仅接收当前问题，还接收之前所有的问答对，使其能够利用先前交流中的信息。

4.2 结果分析实现

笔者使用以下脚本对实验结果进行分析：

▼ 代码块

```

1  def analyze_experiment_results(input_file: str, output_file: str):
2      """分析实验结果并保存详细报告"""
3      # 加载实验结果

```



```

4     results = load_experiment_results(input_file)
5
6     # 将结果转换为DataFrame
7     df = create_analysis_dataframe(results)
8
9     # 重新计算总正确率
10    overall_accuracy = df["correct"].mean()
11
12    # 计算F1分数
13    f1_scores = calculate_f1_scores(df)
14
15    # 创建混淆矩阵
16    cm, classes = create_confusion_matrix(df)
17
18    # 计算每轮的正确率
19    round_accuracy = calculate_round_accuracy(df)
20
21    # 计算每个谜题的F1分数
22    puzzle_f1_scores = {}
23    for puzzle_title in df["puzzle_title"].unique():
24        puzzle_df = df[df["puzzle_title"] == puzzle_title]
25        puzzle_f1_scores[puzzle_title] = calculate_f1_scores(puzzle_df)
26
27    # 提取错误案例
28    error_cases = extract_error_cases(results)
29
30    # 创建分析报告
31    analysis_report = {
32        "overall_accuracy": overall_accuracy,
33        "f1_scores": f1_scores,
34        "confusion_matrix": cm.tolist(),
35        "confusion_matrix_classes": classes,
36        "round_accuracy": round_accuracy,
37        "puzzle_f1_scores": puzzle_f1_scores,
38        "error_cases": error_cases
39    }
40
41    # 保存分析报告
42    with open(output_file, "w", encoding="utf-8") as f:
43        json.dump(analysis_report, f, ensure_ascii=False, indent=2)
44    return analysis_report

```

分析脚本计算了多种评估指标，包括准确率、F1值、混淆矩阵和轮次准确率，并提取了错误案例供进一步分析。

5. 实验结果

5.1 整体准确率

表1和图1显示了三个模型在两组实验中的整体准确率。

模型	实验1（无对话历史）	实验2（有对话历史）	提升
Claude 3.7	92.83%	96.62%	0.0379
GPT-4o	94.94%	94.09%	-0.85%
ChatGLM4	84.81%	92.83%	0.0802

表1: 不同模型在有无对话历史条件下的准确率比较

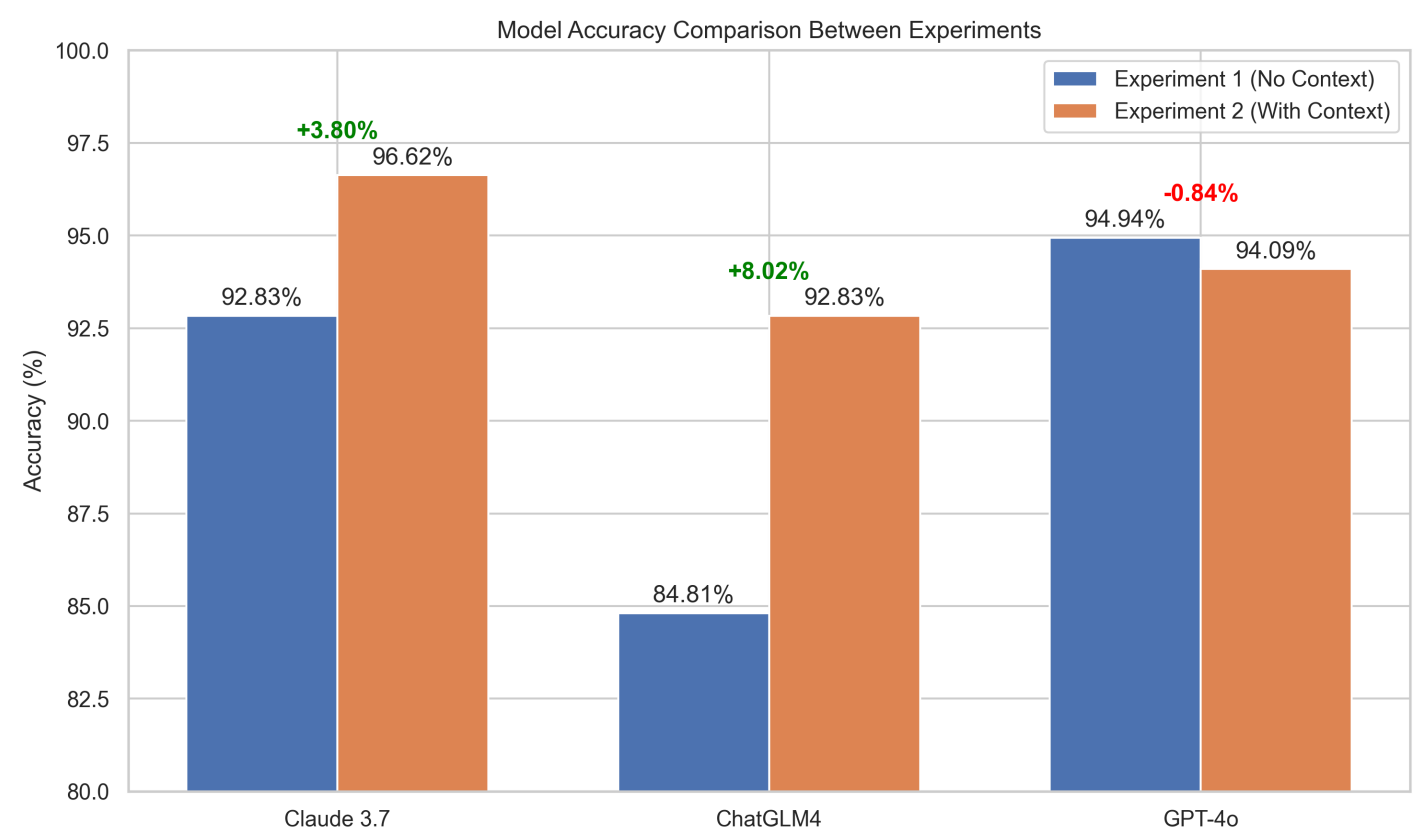


图1: 不同模型在有无对话历史条件下的准确率比较

- 从结果可以看出：
- 在没有对话历史的情况下，GPT-4o表现最佳，准确率为94.94%，其次是Claude 3.7（92.83%）和 ChatGLM4（84.81%）。
 - 添加对话历史后，Claude 3.7的表现显著提升，准确率达到96.62%，成为三个模型中表现最佳的。

3. ChatGLM4从对话历史中获益最多，准确率提升了8.02个百分点，这表明对话历史对性能较弱的模型帮助更大。
4. 有趣的是，GPT-4o在有对话历史的实验中表现略有下降，这可能表明它在处理较长上下文时存在挑战。

5.2 F1值比较

表2展示了三个模型在两组实验中各类别回答的F1值。

模型	实验	"是" F1	"不是" F1	"无关" F1	宏平均F1
Claude 3.7	实验1	0.96	0.6667	0	0.5422
Claude 3.7	实验2	0.9788	0.8085	1	0.9291
GPT-4o	实验1	0.974	0.8085	0.5	0.7608
GPT-4o	实验2	0.9693	0.766	0	0.5784
ChatGLM4	实验1	0.9668	0.72	1	0.8956
ChatGLM4	实验2	0.9619	0.72	0	0.5606

表2: 不同模型在有无对话历史条件下的F1值比较

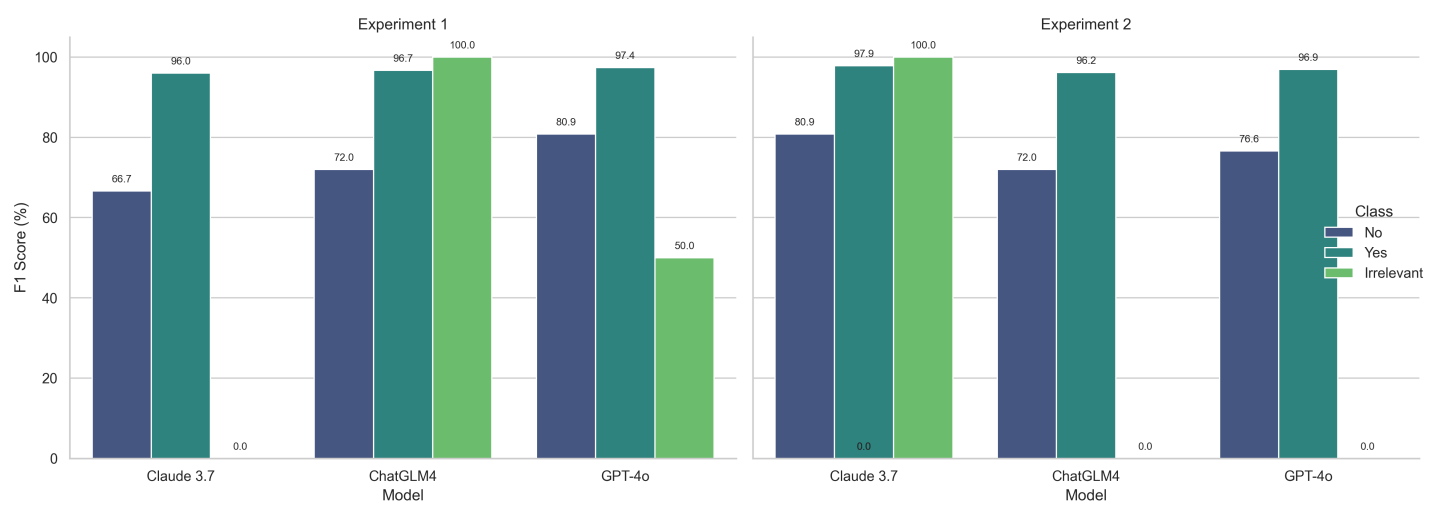


图2: 不同模型在有无对话历史条件下的F1值比较

- 从F1值结果可以发现：
- 所有模型在处理"是"类别的问题上表现较好，F1值普遍在0.96以上。
 - "不是"类别的问题对所有模型都是较大的挑战，F1值明显低于"是"类别。
 - "无关"类别的问题数量较少，模型表现不稳定，F1值变化较大。

4. Claude 3.7在有对话历史的情况下，各类别F1值都有提升，特别是"无关"类别的F1值从0提升到1.0。

5.3 轮次准确率

图1展示了模型在各轮对话中的准确率变化。

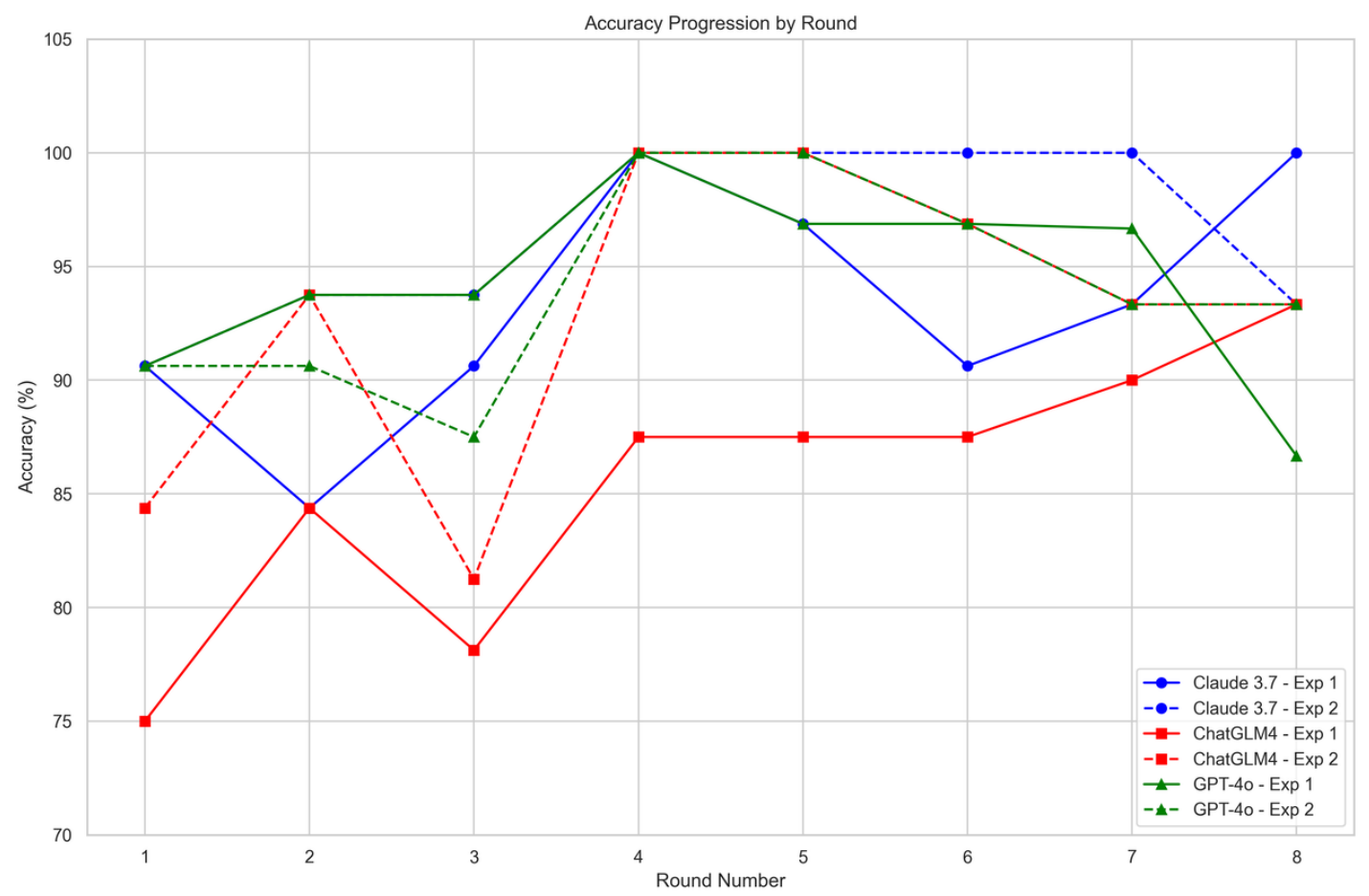


图1: 各模型在不同轮次的准确率变化

- 从轮次准确率图可以看出：
- 1. 在早期轮次（尤其是第1-3轮），模型准确率相对较低，这可能是因为上下文信息较少，推理难度较大。
 - 2. 随着对话的进行，所有模型的准确率总体呈上升趋势，这表明模型能够从对话中积累信息。
 - 3. 在实验2（有对话历史）中，模型在中后期轮次（第4-7轮）的准确率普遍高于实验1，特别是 Claude 3.7和ChatGLM4。
 - 4. 在第8轮（最后一轮），准确率出现了一定程度的下降，这可能是因为最后一轮问题通常是对整个谜底的总结性验证，模型需要综合判断用户的回答是否构成完整的汤底，复杂度较高。

5.4 谜题难度分析

图2展示了不同谜题的难度，以模型在该谜题上的平均F1值衡量。

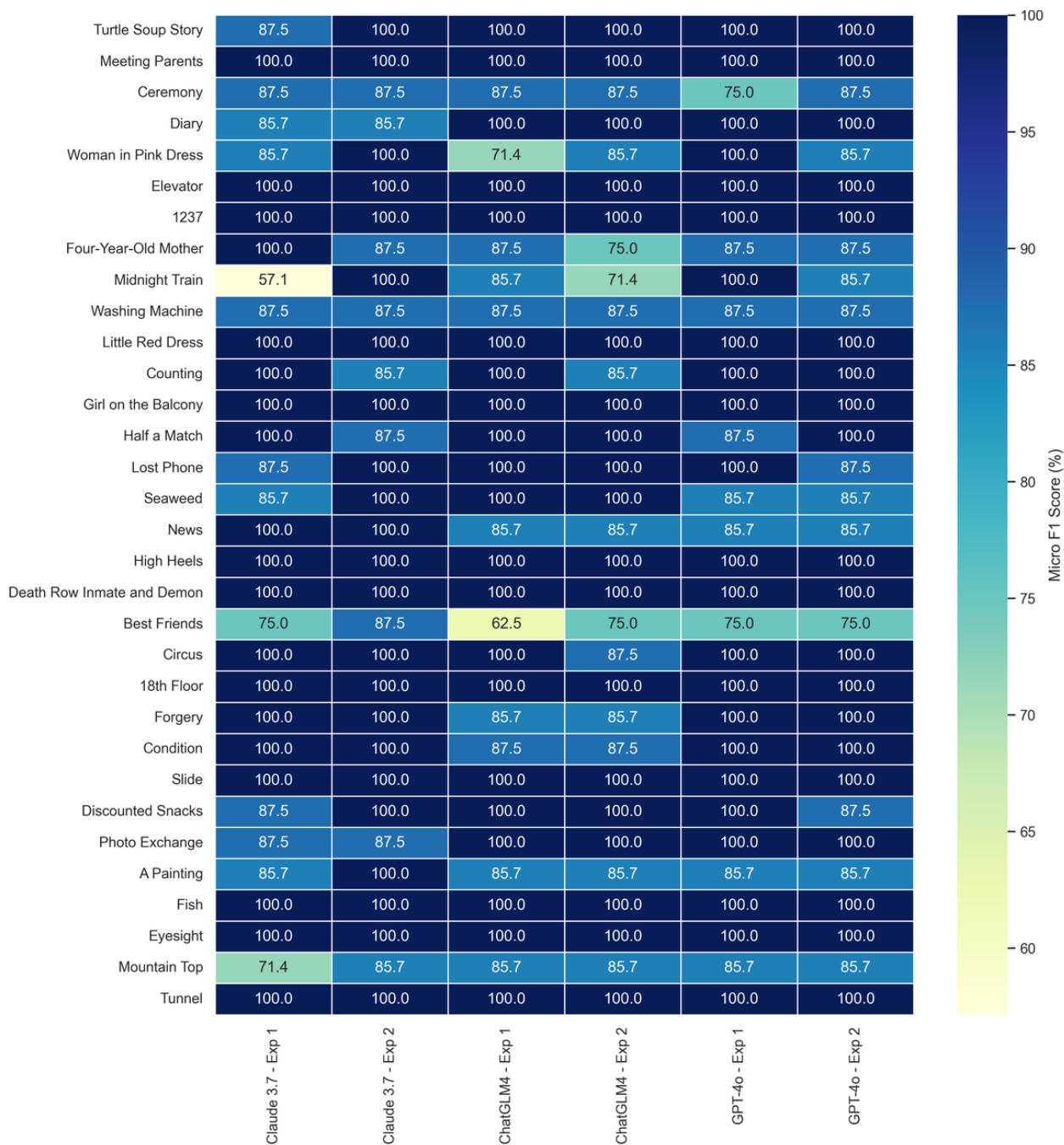


图2: 不同谜题的难度比较

从谜题难度分析可以看出：

1. "午夜列车"、"山顶"、"闺蜜"和"四岁的妈妈"是最具挑战性的谜题，三个模型在这些谜题上的表现都相对较差。
2. "电梯"、"半根火柴"、"视力"等谜题相对简单，模型表现较好。
3. 在有对话历史的条件下，模型在难题上的表现普遍有所提升，特别是在"山顶"和"闺蜜"谜题上。

5.5 混淆矩阵分析

图3展示了各模型在两组实验中的混淆矩阵。

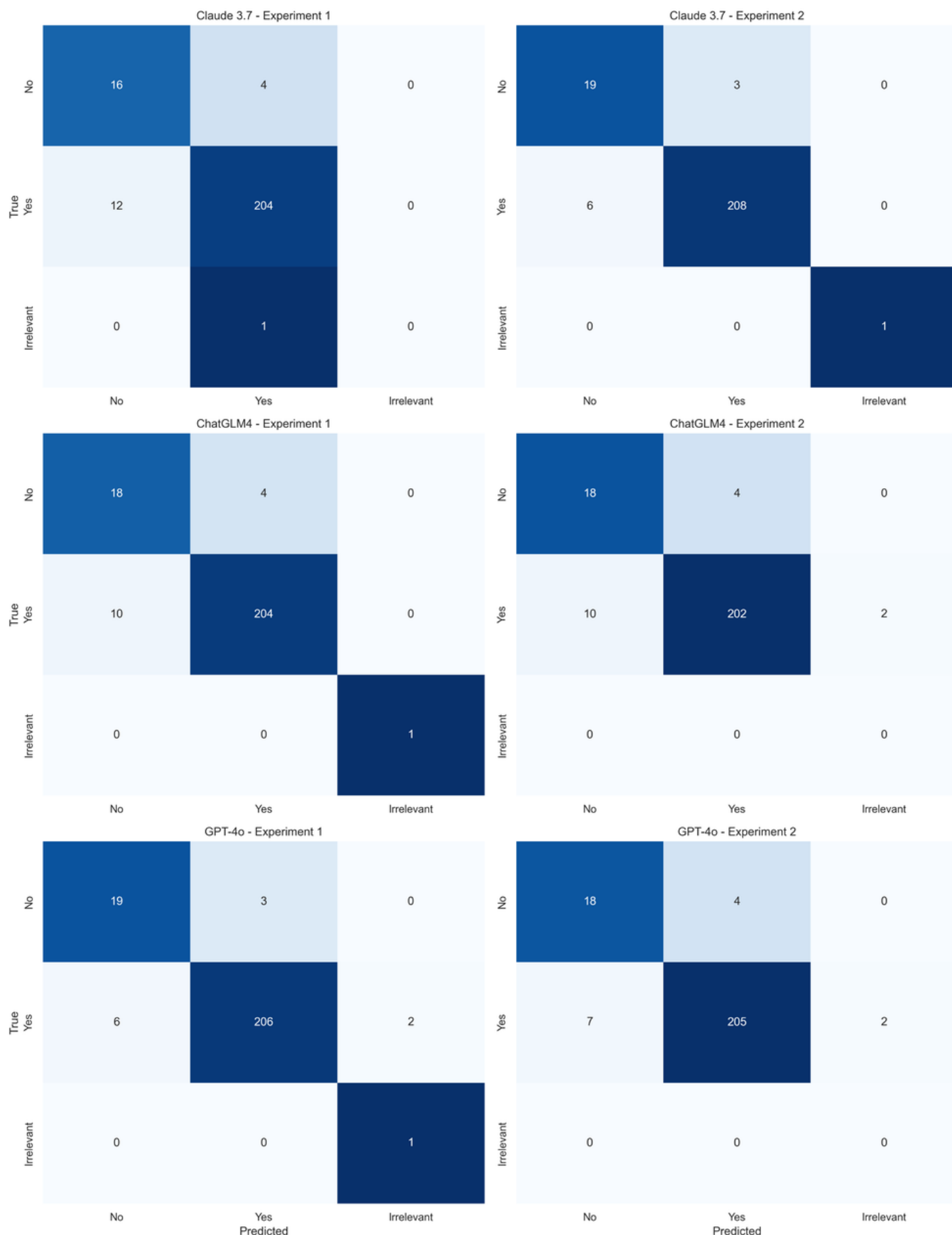


图3: 混淆矩阵比较

从混淆矩阵可以观察到:

1. 所有模型在判断"是"类问题时表现良好, 错误率较低。

- 2. 模型普遍倾向于将"不是"错误分类为"是"，而较少将"是"错误分类为"不是"。
- 3. "无关"类别的问题数量很少，模型对此类问题的处理能力难以准确评估。
- 4. 在有对话历史的实验中，所有模型在处理"不是"类别问题时的准确率都有提升，误分类为"是"的情况减少。

6. 错误样例分析

通过分析实验中的错误案例，笔者发现了一些模式化的错误类型，这些错误揭示了大模型在文本蕴含任务中面临的挑战。

6.1 社会关系推理错误

在"闺蜜"谜题中，所有模型都在判断关于社会关系的问题时出现了错误。例如：

示例1: 社会关系推理错误

▼ 代码块

- 1 谜题：闺蜜
- 2 汤面：托马斯第一次和老婆去老婆闺蜜家，结果回来老婆就要和他离婚了，请问为什么？
- 3 汤底：托马斯老婆看见老公的手机自动连上了闺蜜家的WiFi。
- 4 问题：托马斯在闺蜜家的行为是导致老婆要离婚的原因吗？
- 5 预期答案：不是
- 6 模型回答：是

在这个例子中，模型错误地推断托马斯的行为是导致离婚的原因，而实际上是手机自动连上WiFi这一现象表明托马斯之前来过闺蜜家。这种错误表明模型在理解隐含的社会关系和情感反应时存在困难。

6.2 空间关系推理错误

在"山顶"谜题中，模型在处理空间关系时出现了错误：

示例3: 空间关系推理错误

▼ 代码块

- 1 谜题：山顶
- 2 汤面：一个人住在山顶的小屋里，半夜听见有敲门声音，但是他打开门却没有看到人，于是去睡了。第二天，有人在山脚下发现死尸一具，请问发生了什么？
- 3 汤底：山顶的小屋的门是悬崖，悬崖下的人好不容易才爬上来，想要敲门求救。一开门，就又被推了下去，最后从山顶上掉下去摔死了。
- 4 问题：住在山顶小屋的人开门时，敲门者是否已经从山顶坠落？
- 5 预期答案：不是
- 6 模型回答：是

这个错误表明模型在理解动作时序和空间位置关系时存在困难，无法正确推断敲门者在门被打开时的位置。

6.3 模糊概念理解推断错误

在"仪式"谜题中，模型在理解抽象概念时出错：

示例4: 因果关系推断错误

▼ 代码块

1

谜题：仪式

2

汤面：最近你的父母总是吵架，你也觉得你的母亲状态有些奇怪，她总是心不在焉，偷偷问你怎么给别人转账.....有时候会在家里无声地挥舞双手像在进行什么仪式。一天晚饭过后，你父亲出门去散步了，你母亲也悄悄出了门，已经起了疑心的你跟在她身后.....啊，原来如此.....

3

汤底：你母亲跳广场舞，你父亲以为她外面有人了，转账是为了买跳舞用的团队服装，手舞足蹈是为了练习舞蹈。而且为了防止扰民，所有的广场舞阿姨都是戴着耳机无声跳舞的，看起来像是在进行什么神秘仪式似的。

4

问题：母亲的异常行为是否与某种经济问题有关？

5

预期答案：是

6

模型回答：不是

模型错误地认为母亲的行为与经济问题无关，而实际上转账是为了购买舞蹈服装，这实际上属于一种经济问题，因此理解上存在问题。

7. 结果解释

笔者的实验结果表明，对话历史对大模型在文本蕴含任务中的表现有显著影响。具体而言：

1. 语境累积效应：随着对话的进行，模型可以累积更多语境信息，在后期轮次中表现更好。这与人类在对话中的认知过程相似，早期获取的信息有助于后期的推理判断。
2. 模型差异化影响：不同模型从对话历史中获益程度不同。ChatGLM4从对话历史中获益最多，Claude 3.7也有明显提升，而GPT-4o反而略有下降。这表明模型架构和预训练方法可能影响其利用对话历史的能力。
3. 错误修正机制：对话历史中的正确信息可以帮助模型在后续轮次中避免类似错误。例如，当模型对谜题的某个方面理解错误时，后续对话中的提问和回答可以帮助纠正这种理解。
4. 任务特性匹配度：海龟汤游戏是一种逐步推理的任务，与多轮对话的性质高度匹配。在类似的渐进式推理任务中，保留对话历史尤为重要。